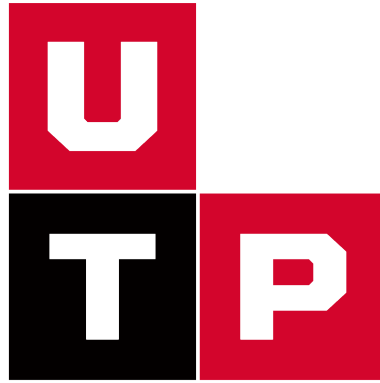




Universidad Tecnológica del Perú

Programación Orientada a Objetos

Proyecto Final
WallpaWawqi - Punto de Venta

Quispe Huisa, Daniel Gonzalo
Huatay Salcedo, Luis Elías
Paredes Alvarez, Paul Gheison
Càrdenas Silva, Jorge Calet
Manrique Gonzales, Liliana Graviela

15 de julio de 2026

Docente: **Guillermo Daniel Vilela Giron**

Índice

1	Introducción	1
2	Problemática	2
3	Solución	2
4	Conclusiones	3
5	Recomendaciones	4

1. Introducción

El presente proyecto desarrolla una plataforma web de punto de venta para *Wallpa Wawqi*, orientada a mejorar la administración de productos, el registro de empleados y la presentación de información del negocio de manera centralizada. La solución fue concebida como un sistema completo, compuesto por un backend en Java, un frontend desarrollado con Astro y una base de datos PostgreSQL, con el objetivo de cubrir tanto la lógica de negocio como la experiencia de uso.

La necesidad principal del proyecto fue reemplazar procesos manuales y dispersos por una aplicación accesible desde la web, capaz de responder con rapidez a las operaciones habituales del negocio. Para ello, se definió una arquitectura separada en capas, donde el backend expone servicios HTTP para autenticación y gestión de productos, mientras que el frontend presenta una interfaz clara para el usuario final y consume dichos servicios de forma ordenada.

En el componente de servidor se empleó Java junto con clases organizadas por responsabilidad, incluyendo modelos de dominio, objetos de acceso a datos y servicios auxiliares para conexión a la base de datos y almacenamiento de imágenes en la nube. En paralelo, el frontend fue diseñado para visualizar la información principal del sistema, permitir el flujo de interacción del usuario y mantener una estructura compatible con el despliegue en Vercel.

Adicionalmente, la persistencia de datos se resolvió con PostgreSQL, alojada en Render.com, lo que permitió mantener la información disponible de forma estable para el backend desplegado en Railway. Esta combinación de tecnologías hizo posible construir una solución modular, funcional y fácil de mantener, alineada con los objetivos académicos del curso y con las necesidades reales del proyecto.

Finalmente, el documento presenta la problemática identificada, la solución implementada, las conclusiones derivadas del desarrollo y las recomendaciones para una versión futura. Las muestras, capturas de pantalla y demostraciones del funcionamiento del sistema se realizarán durante la exposición, por lo que en esta versión se han dejado espacios reservados para incorporar dichas evidencias.

2. Problemática

El desarrollo de un sistema de punto de venta para *Wallpa Wawqi* surgió de la necesidad de centralizar el registro, la administración y la consulta de la información del negocio en una sola plataforma. Antes de este proyecto, la gestión de productos, sesiones de empleados y actualización del inventario dependía de procesos manuales que podían generar demoras, errores de captura y una disponibilidad limitada de la información en tiempo real.

Además, el negocio requería una solución accesible desde la web, capaz de almacenar datos de forma persistente y permitir operaciones de creación, lectura, actualización y eliminación sobre los productos. También se necesitaba una interfaz clara para el usuario final, con una estructura que facilitara el uso cotidiano sin sacrificar orden ni estabilidad en la gestión interna.

Otro aspecto importante fue la necesidad de separar responsabilidades dentro del sistema. Si la lógica de negocio, la persistencia y la presentación se mantenían mezcladas, el mantenimiento posterior del proyecto resultaría más complejo. Por ello, desde el inicio se planteó una arquitectura modular que permitiera identificar con claridad las funciones de cada componente.

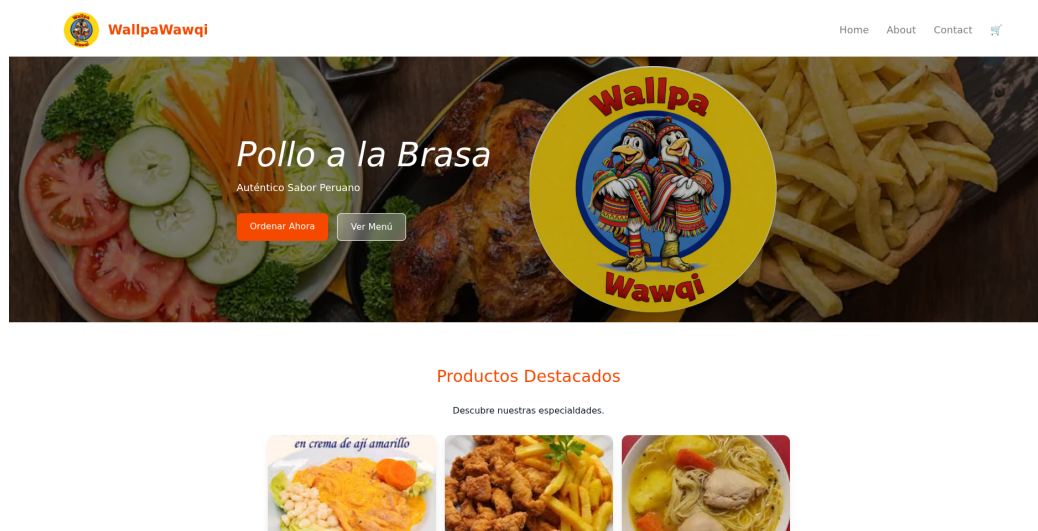


Figura 1: Referencia visual de la situación inicial del sistema. Se observa como no hay un sistema de login, orden de compra o registro de producto.

3. Solución

Para responder a esta problemática se implementó una arquitectura dividida en backend y frontend. El backend fue desarrollado en Java, utilizando una estructura por capas con clases de dominio, acceso a datos y servicios auxiliares. En `ProductoDAO` y `EmpleadoDAO` se centraliza el acceso a la base de datos PostgreSQL, mientras que `DatabaseConnection` gestiona la conexión con el motor de datos y `CloudinaryService` se encarga de almacenar las imágenes de los productos en la nube.

La aplicación expone servicios HTTP mediante `HttpServer` para las operaciones de autenticación, consulta, creación, actualización y eliminación de productos. Además, se integraron clases como `EmpleadoSesion`, `Categoria`, `Inventario` y las subclases de personal para representar de forma más clara la estructura del negocio. Esto permite que cada entidad mantenga una responsabilidad concreta dentro del sistema.

Por su parte, el frontend fue construido con Astro y desplegado en Vercel, ofreciendo una interfaz moderna para visualización de productos, registro, edición y pedidos. Esta separación permite que el backend se ejecute en Railway y que la base de datos PostgreSQL se mantenga en Render.com, logrando una distribución adecuada de servicios y una mejor organización del proyecto.

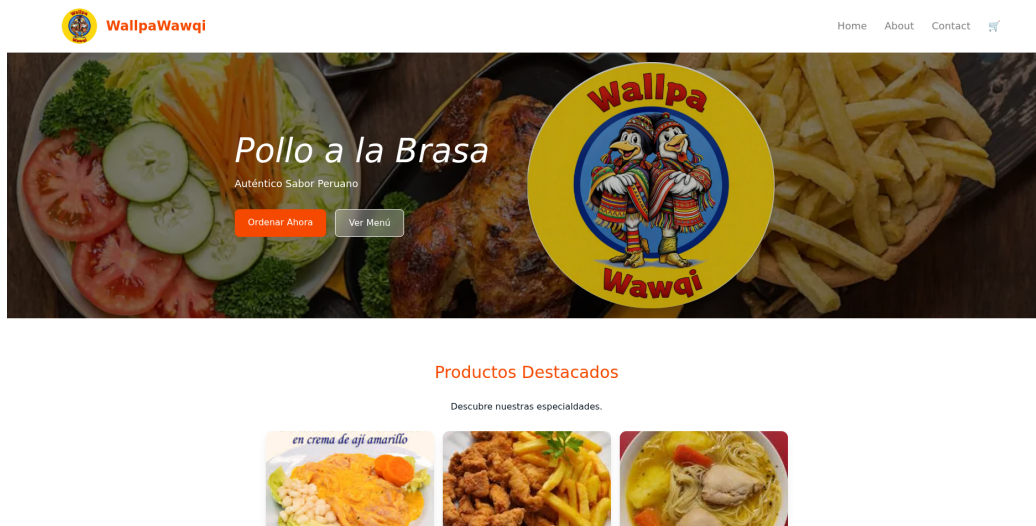


Figura 2: Vista principal del frontend del sistema.

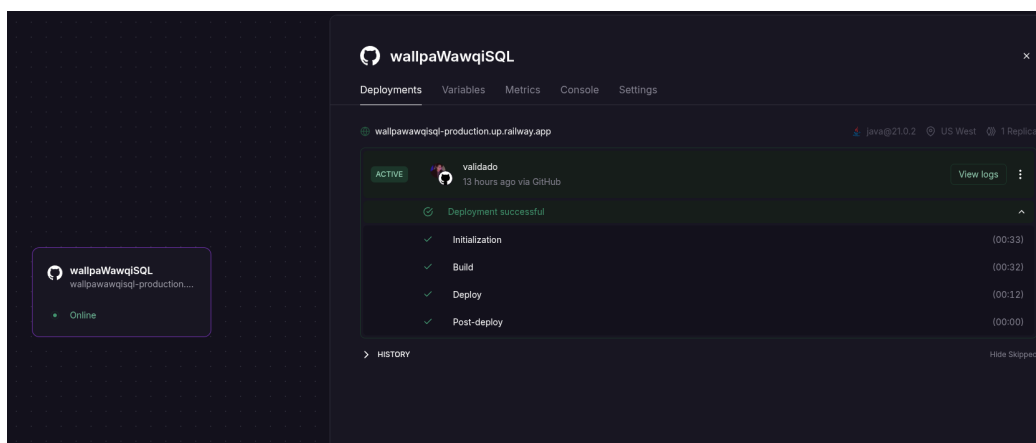


Figura 3: Referencia visual del backend en funcionamiento.

4. Conclusiones

El proyecto permitió consolidar una solución funcional que integra persistencia de datos, servicios web y una interfaz web responsiva para la gestión de un sistema de punto de venta. La distribución entre backend, frontend y base de datos facilitó una implementación ordenada, manteniendo cada tecnología enfocada en una función específica dentro del sistema.

Asimismo, el uso de PostgreSQL, Railway, Vercel y Render.com demuestra que es posible desplegar una aplicación completa con componentes desacoplados y accesibles desde la web. En términos académicos, el proyecto puso en práctica principios de programación orientada a objetos, comunicación cliente-servidor y organización modular del software.

El trabajo también evidenció la importancia de definir desde el inicio una estructura clara para los modelos, los servicios y el acceso a datos. Esa decisión facilitó el desarrollo posterior y permitió mantener una base técnica comprensible para futuras mejoras.

5. Recomendaciones

Se recomienda ampliar el sistema incorporando autenticación más robusta, validaciones adicionales en el frontend y control de errores más detallado en el backend para mejorar la seguridad y la experiencia de uso. También sería conveniente agregar reportes de ventas, categorías dinámicas, historial de operaciones y roles de usuario con permisos diferenciados.

De igual forma, se sugiere optimizar la interfaz para dispositivos móviles y mantener una documentación técnica actualizada de la arquitectura desplegada. Las muestras, capturas de pantalla y demostraciones del funcionamiento del sistema se realizarán durante la exposición para evidenciar el alcance real de la solución implementada.

Finalmente, se recomienda conservar los espacios reservados para figuras hasta contar con las capturas definitivas, de modo que el documento final mantenga coherencia visual y respete la estructura prevista para la presentación.